CLOUD-08: Multi-Cloud Observability Patterns

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 8 of 8 | **Created:** March 2026 | **Last Updated:** 11/11/2026

Overview

This notebook covers strategies for building unified observability across multiple cloud providers with Dynatrace. You will learn how to create cross-cloud dashboards, establish consistent naming conventions, build unified alerting, compare costs across providers, and implement governance patterns for multi-cloud environments.

Table of Contents

1. Multi-Cloud Challenges
2. Unified Entity Model
3. Cross-Cloud Comparison Queries
4. Consistent Naming Conventions
5. Unified Health Dashboards
6. Multi-Cloud Alerting
7. Cost Comparison Patterns
8. Governance Across Providers
9. Summary

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:metrics:read`, `storage:entities:read` + `storage:smartscape:read`, `storage:logs:read`, `storage:buckets:read` (Grail IAM permissions)
Cloud Integrations	At least two cloud providers configured (AWS, Azure, or GCP)
Prior Knowledge	CLOUD-01 through CLOUD-06

1. Multi-Cloud Challenges

Organizations running workloads across multiple cloud providers face these observability challenges:

Challenge	Description
Terminology mismatch	AWS "instances", Azure "VMs", GCP "instances" are all compute but named differently

Challenge	Description
Metric inconsistency	CPU metrics have different units, granularity, and collection methods
Entity fragmentation	Same application may span multiple providers with no inherent linking
Alert fatigue	Separate alerting per provider leads to duplicate or conflicting alerts
Cost opacity	Each provider has different billing models, making comparison difficult
Skill silos	Teams specialize in one provider and lack visibility into others

How Dynatrace Helps

Dynatrace provides a **unified entity model** that normalizes cloud resources across providers:

Dynatrace Concept	AWS	Azure	GCP
`dt.entity.host`	EC2 Instance	Virtual Machine	Compute Instance
`dt.entity.service`	ECS Service / Lambda	App Service	Cloud Run / GKE Service
`dt.entity.kubernetes_cluster`	EKS	AKS	GKE
Tags	AWS Tags	Azure Tags	GCP Labels
`dt.host.cpu.usage` (OneAgent — the same key on every provider)	CloudWatch CPU is a separate key under `cloud.aws.*`	Azure Monitor CPU is a separate key under `cloud.azure.*`	Cloud Monitoring CPU is a separate key under `cloud.gcp.*`

2. Unified Entity Model

Dynatrace normalizes cloud entities into a unified model. This enables cross-cloud queries using the same DQL regardless of provider.

Cross-Cloud Entity Types

Entity Type	Covers
`dt.entity.host`	EC2, Azure VM, GCE instances (with OneAgent)
`dt.entity.service`	Any auto-detected service across all providers
`dt.entity.process_group`	Processes across all hosts/containers
`dt.entity.kubernetes_cluster`	EKS, AKS, GKE clusters
`dt.entity.kubernetes_node`	Worker nodes across all K8s providers

Cloud-Specific Entity Types

Some entities are provider-specific and require separate queries:

AWS	Azure	GCP
`dt.entity.ec2_instance`	`dt.entity.azure_vm`	`dt.entity.cloud:gcp:gce_instance`
`dt.entity.aws_lambda_function`	`dt.entity.azure_web_app`	`dt.entity.cloud_application`
`dt.entity.relational_database_service`	`dt.entity.azure_sql_database`	`dt.entity.cloud:gcp:cloudsql_database`

> **Sources:** AWS integration (DT docs), Azure Native Dynatrace Service (DT docs), Google Cloud integration (DT docs) — the provider-specific entity coverage behind both tables. Every entity type in this section was confirmed present in `dt.semantic_dictionary.models` on 08/27/2026 (the two GCP compute/database types re-checked 09/24/2026). **Dictionary:** note the grain distinction — `dt.entity.kubernetes_node` (worker nodes, 30 on the validation tenant) is **not** interchangeable with `dt.entity.cloud_application_instance` (pods, 2,611 over the same window); substituting one for the other over-counts by roughly 87×.

3. Cross-Cloud Comparison Queries

Compute Resource Count by Provider

```
// Compare compute resource counts across cloud providers (Smartscape)
// smartscapeNodes takes a list of node types. Without from: it returns nodes seen in the default
// window (the last 2 hours), which is the current inventory; from:-7d also counts instances that
// were replaced in the week (85 vs 1,439 EC2 nodes on the validation tenant, 10/02/2026).
smartscapeNodes {"AWS_EC2_INSTANCE", "AWS_LAMBDA_FUNCTION",
                 "AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES", "AZURE_MICROSOFT_WEB_SITES"}
| summarize {resource_count = count()}, by:{type}
| fieldsAdd provider = if(startsWith(type, "AWS_"), then: "AWS", else: "Azure")
| sort provider asc, resource_count desc

// Classic alternative (dt.entity.* is deprecated but still functional): on Clouds-app connections
// the classic types under-count or return nothing for the same estate (validation tenant
// 09/24/2026: EC2 6 classic vs 25 Smartscape; Lambda 2 vs 34; Azure VM 1 vs 8; Web App 0 vs 2),
// so use dt.entity.ec2_instance / azure_vm / aws_lambda_function / azure_web_app with from:-7d
// only where the classic AWS/Azure integration is the source.
```

Unified Host CPU Usage (All Providers)

```
// Top 10 hosts by CPU usage across all cloud providers
timeseries avgCpu = avg(dt.host.cpu.usage), from:-1h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| sort avgCpuValue desc
| limit 10
```

Kubernetes Cluster Comparison

```
// List all Kubernetes clusters (EKS, AKS, GKE)
fetch dt.entity.kubernetes_cluster, from:-7d
| fields id, entity.name, tags
| sort entity.name asc

// Time range required (corrected 08/12/2026): dt.entity.* is an event-LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without an explicit from:
// this under-counted against the notebook default window and still looked like a valid answer.
// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fieldsKeep id, name, tags
//   | sort name asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer entities than
// the classic entity store; for a pre-migration inventory keep the classic query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via getNodeField).
```

Cross-Cloud Problem Analysis

```
// detected problems from the last 7 days across all infrastructure
fetch dt.davis.problems, from:-7d
| filter event.status == "CLOSED"
| summarize {problem_count = count(), avg_duration_hours = avg(resolved_problem_duration / 1h)}, by:
{event.category}
| sort problem_count desc
```

Unified Log Analysis

```
// Error log count by source across all providers in the last 6 hours
fetch logs, from:-6h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel == "ERROR" misses them
| filter status == "ERROR"
| summarize error_count = count(), by:{log.source}
| sort error_count desc
| limit 15
```

4. Consistent Naming Conventions

A naming standard is critical for multi-cloud environments. Apply consistent conventions across all providers.

Recommended Tag Schema

Tag Key	Purpose	Example Values
`cloud-provider`	Identify the source provider	`aws`, `azure`, `gcp`
`environment`	Deployment environment	`prod`, `staging`, `dev`
`team`	Owning team	`platform`, `backend`, `data`
`application`	Application name	`checkout`, `catalog`, `auth`
`cost-center`	Finance allocation	`CC-1234`, `engineering`
`region`	Normalized region name	`us-east`, `eu-west`, `ap-south`

Region Normalization

Cloud providers use different region naming:

Normalized	AWS	Azure	GCP
`us-east`	`us-east-1`	`eastus`	`us-east1`
`us-west`	`us-west-2`	`westus2`	`us-west1`
`eu-west`	`eu-west-1`	`westeurope`	`europe-west1`
`ap-south`	`ap-southeast-1`	`southeastasia`	`asia-southeast1`

Use a normalized `region` tag in Dynatrace to enable cross-cloud region-based queries.

> **Sources:** Tags (DT docs) — the tagging model these conventions target, Central enrichment rules (DT docs) — normalizing provider tags into common dimensions centrally, which is the mechanism that makes a cross-cloud schema enforceable. **Derived:** the specific tag keys and the

> region-normalization map are this entry's convention, not a Dynatrace or cloud-provider standard — adopt or replace them wholesale, but do it once.

5. Unified Health Dashboards

Build dashboards that show health across all providers in a single view.

Dashboard Components

Component	Query Type	Purpose
Infrastructure health	`timeseries avg(dt.host.cpu.usage)`	CPU/memory across all hosts
Active problems	`fetch dt.davis.problems`	Open problems across all infrastructure
Service health	`fetch spans` with error rate	Service-level error rates
Log errors	`fetch logs` with `status == "ERROR"`	Error log trends
Kubernetes health	`timeseries dt.kubernetes.*`	Container metrics across K8s clusters

Active Problems Across All Infrastructure

```
// Active problems grouped by category
fetch dt.davis.problems, from:-24h
| filter event.status == "ACTIVE"
| summarize active_count = countDistinct(event.id), by:{event.category}
| sort active_count desc
```

Service Error Rate (Unified)

```
// Service-level error rates from spans in the last hour.
//
// span.status_code values are LOWERCASE. The uppercase literal matches nothing:
// on a live tenant `== "ERROR"` matched 0 of 733,688 spans while `== "error"`
// matched 19,932 in the same window (08/27/2026). The query still returns rows,
// so the wrong literal reports 0% error rate for every service and nothing errors.
//
// Group by dt.service.name (corrected 09/24/2026): service.name is the OTel resource attribute and
// was null on 75,781 of 82,406 server spans in one hour (09/24/2026), so grouping by it put most of
// the estate in one null bucket. dt.service.name is the detected service and was set on all of them.
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {total = count(), errors = countIf(span.status_code == "error")}, by:{dt.service.name}
| fieldsAdd error_pct = errors * 100.0 / total
| filter total &gt; 10
| sort error_pct desc
| limit 15
```

6. Multi-Cloud Alerting

Alerting Strategy

Level	Scope	Alert Type	Example
Platform	All providers	Dynatrace Intelligence anomaly	Unexpected CPU spike on any host
Provider	Single cloud	Metric threshold	AWS Lambda error rate > 5%
Application	Cross-cloud	Custom metric	Checkout service p95 > 2s
Compliance	All providers	Configuration	Untagged resources detected

Best Practices

Practice	Description
Use Dynatrace Intelligence	Let Dynatrace auto-detect anomalies across all infrastructure
Normalize severity	Map provider-specific severities to a unified scale
Route by team, not provider	Alert the application team regardless of which cloud has the issue
Deduplicate	Dynatrace Intelligence automatically correlates related issues across providers
Define SLOs at the service level	SLOs should be cloud-agnostic

7. Cost Comparison Patterns

While Dynatrace is not a cost management tool, you can use monitoring data to inform cost decisions.

Resource Utilization Comparison

```
// Host utilization over the last 24 hours - identify underutilized hosts
timeseries avgCpu = avg(dt.host.cpu.usage), from:-24h, by:{dt.entity.host}
| fieldsAdd avgCpuValue = arrayAvg(avgCpu)
| filter avgCpuValue &amp;lt; 10
| sort avgCpuValue asc
| limit 20
```

Cost Optimization Indicators

Indicator	What It Means	Action
CPU < 10% sustained	Overprovisioned instance	Rightsize or terminate
Memory < 20% sustained	Overprovisioned memory	Rightsize instance type
Lambda avg duration near timeout	Function at risk	Optimize code or increase timeout
Zero traffic services	Unused resources	Decommission candidates
K8s pods with no requests/limits	Uncontrolled resource usage	Enforce resource quotas

```
// Kubernetes namespaces with highest resource consumption (cost proxies)
timeseries nsCpu = sum(dt.kubernetes.container.cpu_usage, rollup: avg), from:-24h, by:{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(nsCpu)
| sort avgCpu desc
| limit 10
```

8. Governance Across Providers

Governance Framework

Governance Area	Dynatrace Implementation
Access control	Segments by cloud provider + environment; IAM policies per team
Tagging compliance	Monitor for untagged entities; alert on non-compliance
Cost allocation	Tag-based DPS/DDU cost attribution per team/application
Security posture	Runtime vulnerability analysis across all monitored hosts
Change tracking	detected events for configuration changes across providers

Tagging Compliance Check

```
// Find hosts with no tags (potential compliance issue)
//
// Corrected 08/12/2026: `tags == ""` compares an ARRAY to a string and is never true, so the
// clause was dead weight — the query only ever tested isNull(tags). Use arraySize() for the
// "present but empty" case. tags is an array of "key:value" strings; to test for a SPECIFIC tag
// use iAny(), e.g. `filter not iAny(startsWith(tags[], "Team:"))`, and remember to pair it with
// isNull(tags) because iAny() over a null array yields null, which `not` does not turn true.
fetch dt.entity.host, from:-7d
| fields id, entity.name, tags
| filter isNull(tags) or arraySize(tags) == 0
| sort entity.name asc
| limit 20

// Time range required (corrected 08/12/2026): dt.entity.* is an event-LOOKBACK view — it returns
// only entities SEEN in the query window, not the standing inventory. Without an explicit from:
// this under-counted against the notebook default window and still looked like a valid answer.

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | fieldsKeep id, name
// Caveat: Smartscape reflects CURRENT live topology and can report fewer entities than
// the classic entity store; for a pre-migration inventory keep the classic query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via getNodeField).
```

Vulnerability Overview

```
// Problem breakdown by category and status over the last 7 days
//
// Corrected 08/12/2026: the old cell filtered `event.category == "SECURITY"`, a value
// dt.davis.problems never carries — the categories are ERROR, AVAILABILITY, CUSTOM_ALERT,
// RESOURCE_CONTENTION and SLOWDOWN. It returned zero rows on a tenant holding 3,800+ problems and
// read as "no security problems", which is a much more reassuring statement than "this query can
// never match". Application-security findings are NOT Davis problems: they live in the
// security.events / vulnerability data objects and are covered by the APPSEC series.
fetch dt.davis.problems, from:-7d
| summarize problem_count = countDistinct(event.id), by:{event.category, event.status}
| sort problem_count desc
```

Multi-Cloud Governance Best Practices

Practice	Description
Enforce consistent tagging	Use automation to reject untagged resources
Create cloud-agnostic SLOs	Define SLOs based on user experience, not infrastructure
Centralize alert routing	Single Dynatrace workflow routes alerts to the right team
Regular utilization reviews	Monthly review of underutilized resources across all providers
Unified runbooks	Runbooks should reference Dynatrace queries, not provider-specific tools

9. Summary

Key Takeaways

- Dynatrace's **unified entity model** normalizes cloud resources across AWS, Azure, and GCP
- **Cross-cloud queries** use the same DQL patterns via shared entity types like `dt.entity.host` and `dt.entity.service`
- **Consistent tagging** (`cloud-provider` , `environment` , `team` , `application`) is the foundation of multi-cloud governance
- **Alerting should be team-based**, not provider-based — route to the application owner regardless of which cloud has the issue
- Use **utilization data** to identify cost optimization opportunities across all providers

Series Complete

This notebook concludes the CLOUD series. For deeper dives into specific areas, explore:

- **K8S series** — Advanced Kubernetes monitoring patterns
- **OPLOGS series** — OpenPipeline log processing
- **IAM series** — Enterprise IAM administration
- **WFLOW series** — Workflows and alert notifications

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.